

Introduction to Dynamic Programming

Problem: Climbing Stairs — Naive Recursion vs. Memoization vs. Tabulation

Course: Advanced Data Structures and Algorithms | Total Marks: 20

1. Problem Description

The Climbing Stairs problem asks: given a staircase of n steps, where each move advances either 1 or 2 steps, in how many distinct ways can the top be reached? The number of ways follows the recurrence $Ways(n) = Ways(n-1) + Ways(n-2)$, with base cases $Ways(0) = 1$ and $Ways(1) = 1$. This assignment implements and compares three algorithmic paradigms for solving this recurrence: naive recursion, top-down memoization, and bottom-up tabulation.

2. Source Code

2.1 Naive Recursive Approach

```
// 1. Naive Recursive Approach
// Follows the recurrence Ways(n) = Ways(n-1) + Ways(n-2) directly,
// with no storage of previously computed results.
unsigned long long climbStairsNaive(int n) {
    if (n <= 1) return 1; // base cases: Ways(0) = 1, Ways(1) = 1
    return climbStairsNaive(n - 1) + climbStairsNaive(n - 2);
}
```

2.2 Memoization (Top-Down DP)

```
// 2. Memoization (Top-Down DP)
// Recursive structure identical to the naive version, but results
// are cached in a vector so each subproblem is solved only once.
unsigned long long climbStairsMemoHelper(int n, vector<long long>& memo) {
    if (n <= 1) return 1;
    if (memo[n] != -1) return memo[n]; // return cached result
    memo[n] = climbStairsMemoHelper(n - 1, memo)
        + climbStairsMemoHelper(n - 2, memo);
    return memo[n];
}

unsigned long long climbStairsMemo(int n) {
    vector<long long> memo(n + 1, -1); // -1 marks "not yet computed"
    return climbStairsMemoHelper(n, memo);
}
```

2.3 Tabulation (Bottom-Up DP)

```
// 3. Tabulation (Bottom-Up DP)
// Iteratively fills a 1D array from the base cases upward,
// avoiding recursion and function-call overhead entirely.
unsigned long long climbStairsTab(int n) {
    if (n <= 1) return 1;
```

```

vector<unsigned long long> dp(n + 1);
dp[0] = 1;
dp[1] = 1;
for (int i = 2; i <= n; i++) {
    dp[i] = dp[i - 1] + dp[i - 2];
}
return dp[n];
}

```

2.4 Benchmarking Utility

Execution time was measured with a high-resolution chrono clock. Fast-running approaches (Memoization and Tabulation) were repeated 2000 times per input size and averaged, to obtain stable readings above the timer's resolution floor; the Naive approach was timed on a single run, since it is already slow enough at large n to dominate any timer overhead.

```

// Benchmark helper using <chrono>
template <typename Func>
double measureTime(Func f, int n, int repeats = 1) {
    auto start = high_resolution_clock::now();
    for (int r = 0; r < repeats; r++) {
        volatile unsigned long long result = f(n);
        (void)result;
    }
    auto end = high_resolution_clock::now();
    duration<double, micro> elapsed = end - start;
    return elapsed.count() / repeats;    // average time in microseconds
}

```

3. Running Time Table

All code was compiled with g++ (C++17, -O2 optimization) and run on the same machine for all three approaches, so relative timings are directly comparable. The Naive Recursive approach was only tested up to $n = 40$, since larger inputs become impractically slow (its cost roughly doubles with each additional step).

n	Naive Recursion (μs)	Memoization (μs)	Tabulation (μs)
10	0.662	0.054	0.021
15	2.674	—	—
20	13.881	0.117	0.026
25	206.120	—	—
30	1516.914	0.251	0.029
35	16337.160	—	—
40	229165.617	0.396	0.031
50	— (not tested)	0.525	0.035
60	— (not tested)	0.663	0.038
70	— (not tested)	0.782	0.042
80	— (not tested)	0.897	0.047
90	— (not tested)	1.061	0.048

Table 1: Measured execution time (microseconds) for each approach across input sizes n . Times for Memoization and Tabulation are averaged over 2000 repetitions; the Naive time is a single-run measurement.

4. Performance Graphs

Figure 1 below plots all three approaches together on a logarithmic Y-axis, which is necessary because the Naive approach's execution time (up to ~229 milliseconds at $n = 40$) is roughly five to six orders of magnitude larger than Memoization or Tabulation at comparable n .

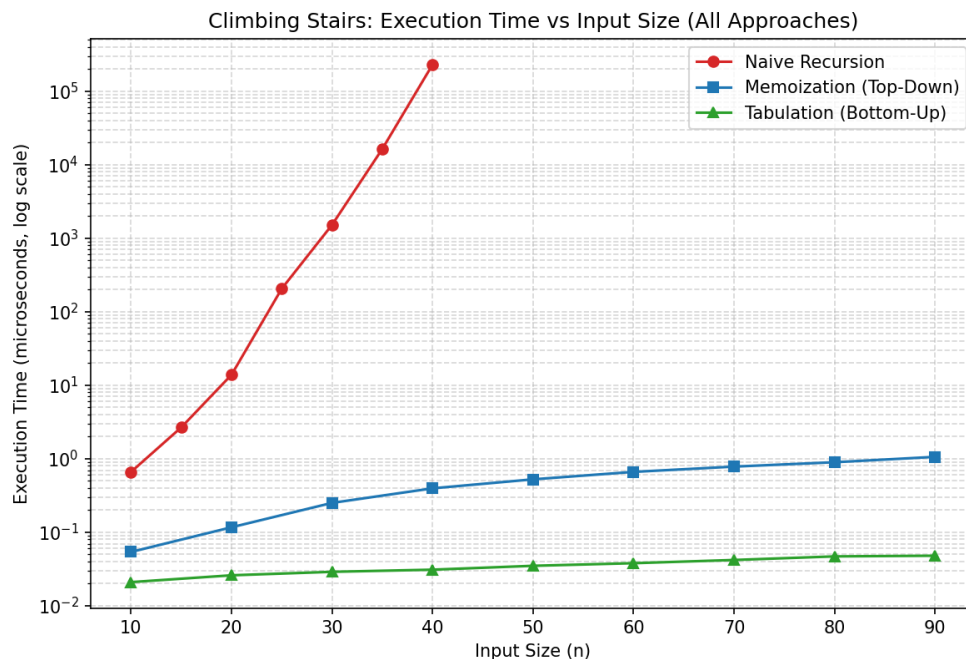


Figure 1: Execution time (log scale) vs. input size for all three approaches.

Figure 2 isolates the Naive Recursive approach on a linear scale, clearly showing its exponential growth curve.

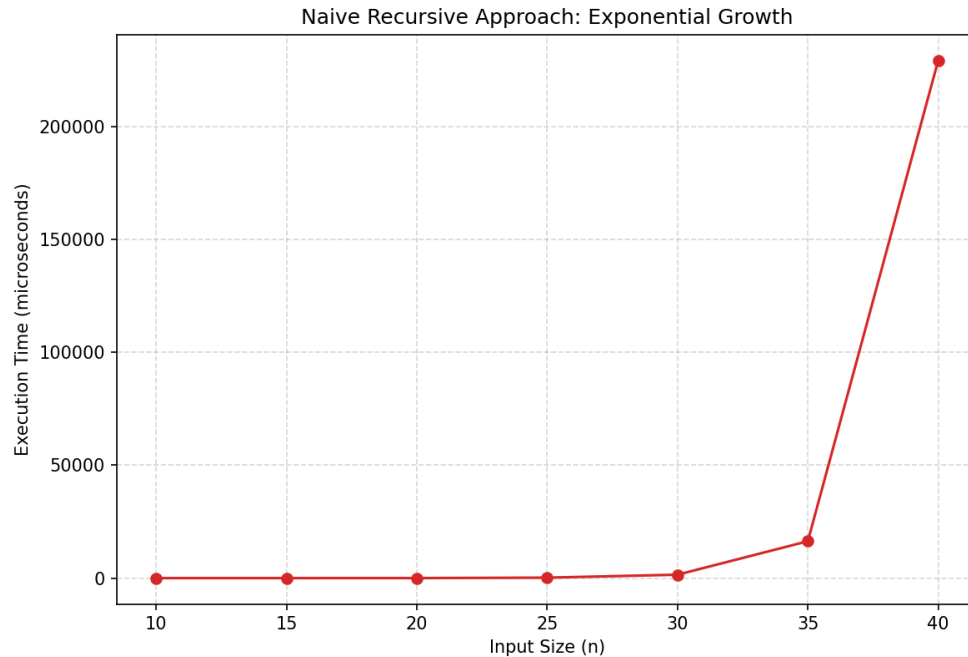


Figure 2: Naive recursion execution time (linear scale) — exponential blow-up.

Figure 3 compares Memoization and Tabulation alone on a linear scale, where both remain fast enough to be readable without a log transform.

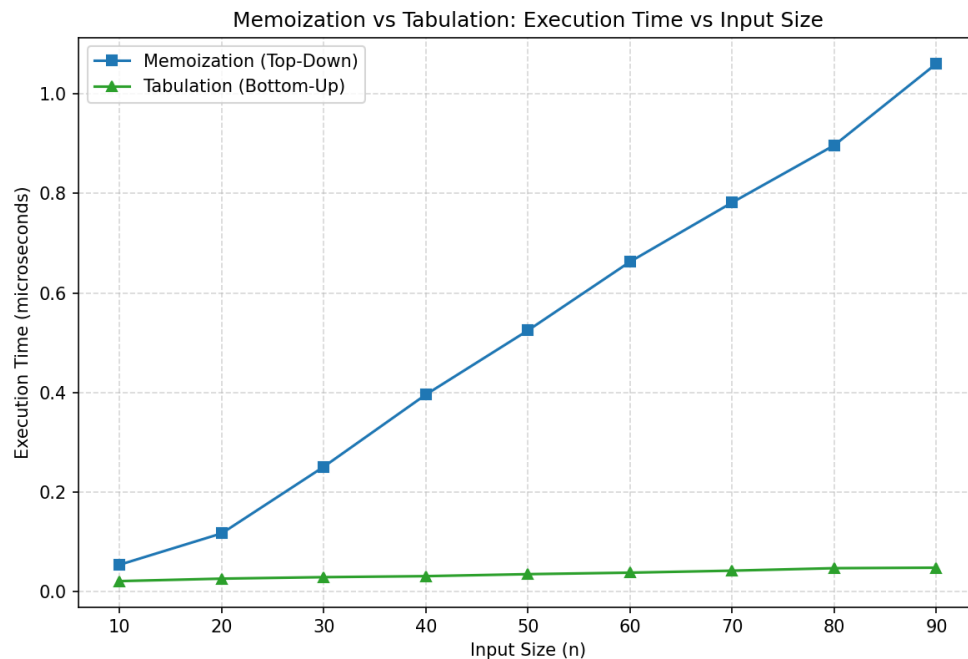


Figure 3: Memoization vs. Tabulation execution time (linear scale).

5. Discussion & Analysis

Why does the naive recursive approach perform so poorly?

The naive recursive approach recomputes the same subproblems many times over. Each call to `climbStairsNaive(n)` branches into `climbStairsNaive(n-1)` and `climbStairsNaive(n-2)`, and those calls branch again, so the same value of, say, `Ways(5)` is recalculated a huge number of times as part of the larger computation. This is the classic 'overlapping subproblems' inefficiency: the call tree grows like a binary tree of depth n , giving roughly $O(2^n)$ total calls (more precisely $O(\phi^n)$, where ϕ is the golden ratio, since each call spawns two smaller calls following the same Fibonacci-like recurrence). The benchmark data confirms this directly — the time roughly doubles or more with each step up in n once n is reasonably large: it takes about 1.5 ms at $n = 30$ but 229 ms at $n = 40$, a jump far too large to be explained by anything but exponential growth. Memoization and Tabulation avoid this entirely by ensuring each subproblem `Ways(k)` is computed exactly once.

Theoretical complexity vs. empirical results

Naive Recursion: time complexity is $O(\phi^n)$ (exponential), and space complexity is $O(n)$ for the recursion call stack (no explicit storage, but stack depth grows linearly with n). This matches the empirical graph, whose curve is a straight line on the log-scale plot with respect to n — a hallmark of exponential growth.

Memoization (Top-Down): time complexity is $O(n)$, since each of the $n+1$ subproblems is computed once and cached; space complexity is $O(n)$ for both the memo array and the recursion call stack. The empirical data shows execution time growing roughly linearly with n (about 1.06 μ s at $n = 90$ versus 0.05 μ s at $n = 10$), consistent with $O(n)$.

Tabulation (Bottom-Up): time complexity is also $O(n)$, since it fills an array of size $n+1$ in a single loop; space complexity is $O(n)$ for the DP array (this can further be optimized to $O(1)$ by keeping only the last two values, though the assignment specifies a 1D array). The measured times again grow roughly linearly with n , from about 0.02 μ s at $n = 10$ to 0.05 μ s at $n = 90$, matching the $O(n)$ prediction.

Top-Down vs. Bottom-Up: is there a difference, and why?

Yes — Tabulation was consistently and noticeably faster than Memoization at every tested value of n , by roughly a factor of 15 to 20 in this benchmark (for example, about 0.031 μ s vs. 0.396 μ s at $n = 40$). Both are $O(n)$ asymptotically, so this is a constant-factor difference rather than a difference in growth rate, but it is large enough to matter in practice. The main reason is overhead: Memoization still relies on recursive function calls, each of which pushes a new stack frame, incurs function-call overhead, and requires a check against the memo table (including the -1 'not computed yet' branch) before doing useful work. Tabulation, in contrast, is a simple iterative loop with no function-call overhead, no recursive stack growth, and very predictable, cache-friendly sequential memory access as it walks through the array. In C++ specifically, these effects are pronounced because function calls and stack frame management carry real, measurable cost even after compiler optimizations, whereas a tight iterative loop can be optimized very effectively (including partial loop unrolling) by the compiler.

Conclusion

Overall, this experiment illustrates the core motivation for dynamic programming: recognizing overlapping subproblems and eliminating redundant work turns an exponential-time algorithm into a linear-time one. Between the two DP strategies, Tabulation edges out Memoization in raw performance in this C++ implementation due to lower constant-factor overhead from avoiding recursion, even though both share the same $O(n)$ asymptotic complexity.